

## 08. 데이터셋 관리 - OMX AI!

---



# Contents

---

## 1 본 자료의 구성

## 1 데이터 시각화

### 1.1 온라인 웹을 통한 데이터 시각화

### 1.2 LeRobot Dataset Visualizer란?

### 1.3 시각화 예시

### 1.4 시각화 화면 구성 요소

### 1.5 에피소드(Episode)란?

### 1.6 녹화된 에피소드를 다시 재생하는 명령어

### 1.7 lerobot-replay 명령어 옵션 설명

### 1.8 시각화를 통해 확인할 리스트

### 1.9 시각화 체크리스트 상세 설명

## 2 LeRobot Dataset Tools로 데이터 관리

### 2.1 데이터셋 관리를 위한 여러 도구들을 제공한다

### 2.2 lerobot-edit-dataset 도구 개요

### 2.3 에피소드 삭제 예시 - 원본 데이터셋 수정

### 2.4 에피소드 삭제 예시 - 원본 데이터셋 유지

### 2.5 에피소드 삭제 옵션 설명

### 2.6 비율 기준으로 데이터셋 분할 (e.g. 80% train, 20% test, 20% val)

### 2.7 특정 에피소드 번호 기준으로 데이터셋 분할

### 2.8 데이터셋 분할 방식 설명

## 2.9 데이터셋 병합(merge)

---

### 2.10 데이터셋 병합 시 주의사항

---

### 2.11 특징 제거

---

### 2.12 특징 제거 활용 설명

---

### 2.13 데이터셋 관리 도구 비교 정리

---

### 2.14 학습 포인트

---

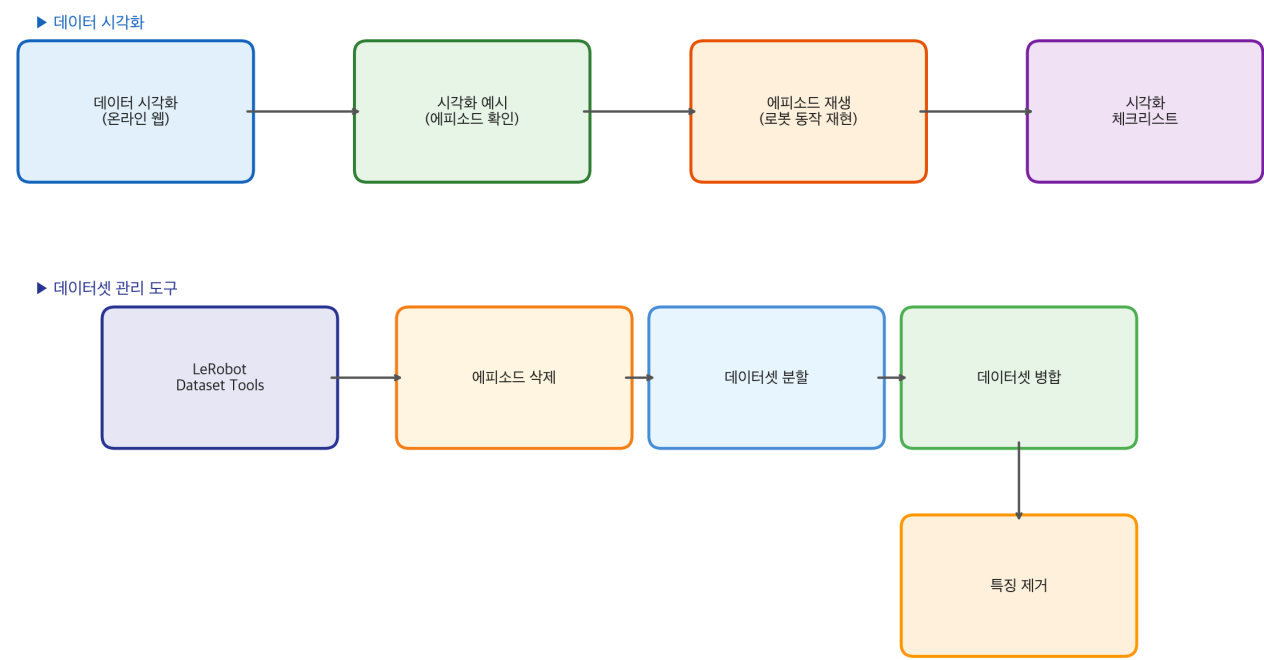
# 08. 데이터셋 관리 - OMX AI!

Exported from Confluence · <https://pinkwink.atlassian.net>

이 페이지는 OMX AI 로봇의 데이터셋 관리 방법을 다룹니다. LeRobot 프레임워크를 활용한 데이터 시각화, 에피소드 재생, 그리고 데이터셋 편집 도구(삭제/분할/병합/특징 제거)의 사용법을 학습합니다.

## 본 자료의 구성

08. 데이터셋 관리 - OMX AI 자료 구성



## 데이터 시각화

### 온라인 웹을 통한 데이터 시각화

- Hub에 업로드되어 있다면, 원하는 **repo id**를 복사해 붙여넣으면 웹에서 데이터셋을 **온라인으로 시각화** ([https://huggingface.co/spaces/lerobot/visualize\\_dataset](https://huggingface.co/spaces/lerobot/visualize_dataset)) 할 수 있다.

# LeRobot Dataset Visualizer

[create & train your own robots](#)

Enter dataset id (e.g. lerobot/vi)

Go

Example Datasets:

lerobot/aloha\_static\_cups\_open

lerobot/columbia\_cairlab\_pusht\_real

lerobot/taco\_play

Explore Open Datasets

LeRobot Dataset Visualizer란?

## LeRobot Dataset Visualizer 개념

- **LeRobot Dataset Visualizer**는 HuggingFace Spaces에서 제공하는 웹 기반 데이터셋 시각화 도구
- 별도의 로컬 환경 설정 없이 브라우저에서 바로 데이터셋을 확인할 수 있음
- **repo id**: HuggingFace Hub에 업로드된 데이터셋의 고유 식별자 (예: **사용자명/데이터셋명** )
- Example Datasets 목록에서 공개 데이터셋을 바로 선택하여 탐색 가능
- **Explore Open Datasets** 버튼으로 다양한 공개 데이터셋을 둘러볼 수 있음

## 시각화 예시

Spaces
 | lerobot/visualize\_dataset
 

like
 356
 

Running

Number of samples/frames: 316

Number of episodes: 5

Frames per second: 30

Episodes:

[Episode 0](#)

[Episode 1](#)

[Episode 2](#)

[Episode 3](#)

[Episode 4](#)

**IMsubin/record-test**  
 episode 0

observation.images.front

**Language Instruction:**

Pick up the Dynamixel Motor

0.76
 

0.76

☒ wrist flex. pos
 

1 / 6

☒ action
 

53.36

☒ observation.state
 

0.76

☒ observation.vel
 

53.36

Space to pause/unpause

↑ / ↓ to previous/next episode

시각화 화면 구성 요소

## 시각화 화면 주요 구성 요소

- 좌측 정보 패널
  - `Number of samples/frames` : 전체 프레임(샘플) 수 — 예시에서 316개
  - `Number of episodes` : 녹화된 에피소드 수 — 예시에서 5개
  - `Frames per second` : 초당 프레임 수(FPS) — 예시에서 30fps
  - 에피소드 목록에서 원하는 에피소드를 클릭하여 전환 가능
- 카메라 뷰: `observation.images.front` — 로봇 전방 카메라의 영상 데이터
- **Language Instruction**: 해당 에피소드의 태스크 지시문 (예: "Pick up the Dynamixel Motor")
- **관절 그래프**: 각 관절의 위치/상태 값이 시간 축으로 시각화됨
  - 좌측 그래프: `observation` (관측값) — 로봇이 실제 읽은 관절 위치
  - 우측 그래프: `action` (동작값) — 로봇에 전달된 제어 명령
- **재생 컨트롤**: 재생/일시정지(Space), 이전/다음 에피소드(◀/▶) 전환 가능

## 에피소드(Episode)란?

### 에피소드(Episode) 개념 정의

- **에피소드(Episode)**: 하나의 태스크 수행을 처음부터 끝까지 녹화한 데이터 단위
- 하나의 에피소드에는 카메라 이미지, 관절 상태, 동작 명령 등이 시간 순서대로 기록됨
- 여러 에피소드를 모아 하나의 **데이터셋(Dataset)**을 구성함
- 에피소드 인덱스(0, 1, 2...)로 개별 에피소드를 식별하고 관리할 수 있음

## 녹화된 에피소드를 다시 재생하는 명령어

- 팔로워 로봇을 에피소드에 기록된 움직임으로 재생하므로, 환경이 유사한지 확인하고 진행
- `--dataset.episode=0` : 원하는 에피소드 인덱스를 입력
- `--dataset.repo_id=${HF_USER}/record-test` : Hub에 올려져 있는 원하는 데이터셋 기입



bash

```
1 source ~/venv/il/bin/activate
2 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[0-9;]*m//g' | awk -F': ' '{print $2}' | xargs)
```

bash

```
1 cd ~/il_ws/src/lerobot && lerobot-replay \
2   --robot.type=omx_follower \
3   --robot.port=/dev/ttyACM0 \
4   --robot.id=omx_follower_arm \
5   --dataset.repo_id=${HF_USER}/record-test \
6   --dataset.episode=0
```

## lerobot-replay 명령어 옵션 설명

### lerobot-replay 주요 옵션

- `--robot.type=omx_follower` : 사용할 로봇 타입을 OMX 팔로워로 지정
- `--robot.port=/dev/ttyACM0` : 로봇이 연결된 시리얼 포트 경로
- `--robot.id=omx_follower_arm` : 로봇의 고유 식별 ID
- `--dataset.repo_id` : HuggingFace Hub의 데이터셋 경로 (사용자명/데이터셋명)
- `--dataset.episode` : 재생할 에피소드의 인덱스 번호 (0부터 시작)

## 시각화를 통해 확인할 리스트

- 에피소드가 성공적으로 완료되었는지
- 각 카메라들과 동작 간의 프레임 드롭이 없는지
- 카메라 프레임에 객체 정보가 모두 담겨있는지
- Task에 중요한 객체에 대한 정보가 잘 들어오는지
- 급격한 로봇의 움직임이 없었는지

## 시각화 체크리스트 상세 설명

## 데이터 품질 검증 체크포인트

- **에피소드 완료 확인:** 태스크가 중간에 끊기지 않고 처음부터 끝까지 정상 녹화되었는지 확인
- **프레임 드롭:** 카메라 영상과 관절 데이터 사이에 시간 동기화가 어긋나거나, 프레임이 누락된 구간이 없는지 확인
  - 프레임 드롭이 있으면 학습 시 잘못된 대응 관계가 생겨 모델 성능이 저하됨
- **객체 정보:** 조작 대상 객체가 카메라 시야(FOV) 안에 있는지 확인
  - 객체가 프레임 밖으로 벗어나면 모델이 시각 정보 없이 동작을 학습하게 됨
- **급격한 움직임:** 관절 그래프에서 급격한 값 변화(스파이크)가 있으면 데이터 품질 문제
  - 텔레오퍼레이션 중 의도치 않은 튜브 현상이 발생했을 수 있음

# LeRobot Dataset Tools로 데이터 관리

## 데이터셋 관리를 위한 여러 도구들을 제공한다

- **에피소드 삭제** - 데이터 세트에서 특정 에피소드를 제거
- **데이터셋 분할** - 하나의 데이터셋을 여러 개의 작은 데이터셋으로 분할
- **데이터셋 병합** - 여러 데이터셋을 하나로 결합
  - 데이터셋은 동일한 특징을 가져야 하며, 에피소드는 지정된 `repo_ids` 순서대로 연결
- **기능 추가** - 데이터 세트에 새로운 기능을 추가
- **특징 제거** - 데이터 세트에서 특정 특징을 제거
- 자세한 API 사용 방법을 자세히 설명하는 예제 스크립트는 `examples/dataset/use_dataset_tools.py` 에서 확인 가능

## lerobot-edit-dataset 도구 개요

## lerobot-edit-dataset CLI 도구

- **lerobot-edit-dataset**: LeRobot에서 제공하는 데이터셋 편집 CLI 명령어
- 하나의 명령어로 삭제, 분할, 병합, 특징 제거 등 다양한 작업을 수행할 수 있음
- `--operation.type` 파라미터로 수행할 작업 유형을 선택
- `--repo_id` : 작업 대상 데이터셋의 HuggingFace Hub 경로
- `--new_repo_id` : 원본을 유지하면서 결과를 새 데이터셋으로 저장할 때 사용

## 에피소드 삭제 예시 - 원본 데이터셋 수정

bash

```
1 source ~/venv/il/bin/activate
2 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[[0-9;]*m//g' | awk -F': ' '{print $2}' | xargs)
3
4 cd ~/il_ws/src/lerobot && lerobot-edit-dataset \
5     --repo_id ${HF_USER}/record-test \
6     --operation.type delete_episodes \
7     --operation.episode_indices "[0, 2, 3]"
```

## 에피소드 삭제 예시 - 원본 데이터셋 유지

Code

```
1 source ~/venv/il/bin/activate
2 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[[0-9;]*m//g' | awk -F': ' '{print $2}' | xargs)
3
4 cd ~/il_ws/src/lerobot && lerobot-edit-dataset \
5     --repo_id ${HF_USER}/record-test \
6     --new_repo_id ${HF_USER}/record-test2 \
7     --operation.type delete_episodes \
8     --operation.episode_indices "[0, 2, 3]"
```

## 에피소드 삭제 옵션 설명

## delete\_episodes 동작 방식

- `--operation.type delete_episodes` : 삭제 모드 지정
- `--operation.episode_indices "[0, 2, 3]"` : 삭제할 에피소드 인덱스 목록을 JSON 배열로 전달
- 원본 수정 모드: `--new_repo_id` 를 지정하지 않으면 원본 데이터셋에서 직접 삭제됨
- 복사 후 삭제 모드: `--new_repo_id` 를 지정하면 원본은 유지하고 새 데이터셋에 결과를 저장
- 삭제 후 나머지 에피소드의 인덱스가 자동으로 재정렬됨

## 비율 기준으로 데이터셋 분할 (e.g. 80% train, 20% test, 20% val)

- 분할 이름에는 제약 조건이 없으며 사용자가 지정 가능
- 결과 데이터 세트는 저장소 ID에 분할 이름이 추가되어 저장  
(ex: `lerobot/pusht_train` , `lerobot/pusht_task1` , `lerobot/pusht_task2` )

### Code

```
1 source ~/venv/il/bin/activate
2 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[[0-9;]*m//g' | awk -F': ' '{print $2}') | xargs)
3
4 cd ~/il_ws/src/lerobot && lerobot-edit-dataset \
5     --repo_id ${HF_USER}/record-test \
6     --operation.type split \
7     --operation.splits '{"train": 0.8, "test": 0.2, "val": 0.2}'
```

## 특정 에피소드 번호 기준으로 데이터셋 분할

### bash

```
1 # 특정 에피소드 번호 기준으로 데이터셋 분할
2 source ~/venv/il/bin/activate
3 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[[0-9;]*m//g' | awk -F': ' '{print $2}') | xargs)
4
5 cd ~/il_ws/src/lerobot && lerobot-edit-dataset \
6     --repo_id ${HF_USER}/record-test \
7     --operation.type split \
8     --operation.splits '{"task1": [0, 1, 2, 3], "task2": [4, 5]}'
```

## 데이터셋 분할 방식 설명

## split 동작 방식

- 비율 기반 분할: `{"train": 0.8, "test": 0.2}` 형태로 비율을 지정하면 에피소드가 자동 배분됨
- 인덱스 기반 분할: `{"task1": [0,1,2,3], "task2": [4,5]}` 형태로 에피소드를 직접 지정
- 결과 데이터셋 이름: 원래 `repo_id` 에 분할 이름이 접미사로 붙음 (예: `record-test_train` )
- 활용 예시:
  - 학습/검증/테스트 데이터 분리 (train/val/test)
  - 서로 다른 태스크의 에피소드를 별도 데이터셋으로 분리

## 데이터셋 병합(merge)

- 여러 데이터셋을 하나의 데이터셋으로 결합
- 각 데이터셋들은 동일한 특징을 가져야 한다.

bash

```
1 source ~/venv/il/bin/activate
2 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[0-9;]*m//g' | awk -F': ' '{print $2}' | xargs)
```

bash

```
1 lerobot-edit-dataset \
2   --repo_id lerobot/pusht_merged \
3   --operation.type merge \
4   --operation.repo_ids "['lerobot/pusht_train', 'lerobot/pusht_val']"
```

## 데이터셋 병합 시 주의사항

## merge 동작 방식과 조건

- `--operation.type merge` : 병합 모드 지정
- `--operation.repo_ids` : 병합할 데이터셋 목록을 Python 리스트 형태로 전달
- 필수 조건: 병합 대상 데이터셋들의 feature(특징) 구조가 동일해야 함
  - 카메라 이름, 관절 수, 데이터 타입 등이 모두 일치해야 정상 병합됨
- 에피소드는 `repo_ids` 에 나열된 순서대로 연결됨
- 병합 결과는 `--repo_id` 로 지정한 새 데이터셋에 저장

## 특징 제거

bash

```
1 source ~/venv/il/bin/activate
2 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[[0-9;]*m//g' | awk -F: ' '{print $2}' | xargs)
```

bash

```
1 lerobot-edit-dataset \
2     --repo_id lerobot/pusht \
3     --operation.type remove_feature \
4     --operation.feature_names "['observation.images.top']"
```

## 특징 제거 활용 설명

### remove\_feature 활용

- `--operation.type remove_feature` : 특징 제거 모드 지정
- `--operation.feature_names` : 제거할 특징 이름 목록 (예: `observation.images.top` )
- 활용 예시:
  - 사용하지 않는 카메라 데이터 제거 (예: 불필요한 상단 카메라)
  - 학습에 필요 없는 센서 데이터를 제거하여 데이터셋 용량 절감
  - 특정 관절 데이터만 남기고 나머지를 제거하여 간소화된 데이터셋 구성

## 데이터셋 관리 도구 비교 정리

| 도구      | operation.type  | 주요 파라미터            | 용도                         |
|---------|-----------------|--------------------|----------------------------|
| 에피소드 삭제 | delete_episodes | episode_indices    | 품질이 낮은 에피소드 제거             |
| 데이터셋 분할 | split           | splits (비율 또는 인덱스) | train/val/test 분리, 태스크별 분리 |
| 데이터셋 병합 | merge           | repo_ids           | 여러 데이터셋을 하나로 통합            |
| 특징 제거   | remove_feature  | feature_names      | 불필요한 센서/카메라 데이터 제거         |

## 학습 포인트

- 데이터 품질이 모델 성능을 결정한다 — 시각화를 통해 반드시 에피소드 품질을 검증할 것
- 원본 보존 습관 — 에피소드 삭제 시 `--new_repo_id` 를 사용하면 원본을 안전하게 보존할 수 있음
- 데이터셋 구조 일치 — 병합 시 feature 구조가 반드시 동일해야 함 (카메라, 관절 수 등)
- 분할 전략 — 비율 기반은 무작위 배분, 인덱스 기반은 의도적 배분이므로 목적에 맞게 선택
- 특징(feature)이란 데이터셋의 각 데이터 항목(카메라 이미지, 관절 상태, 동작 명령 등)을 의미



## PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

[https://www.youtube.com/@pinklab\\_studio](https://www.youtube.com/@pinklab_studio)